Documentation Projet : VISION

Par : Hélie BENNACEUR , Houssem CHIHEB,
Matisse TESTARD, Anis GRABSI
& Maël MOINDI YANGBANDA

Table des matières :

1. Introduction
2. Objectifs du projet
3. Architecture du système
4. Fonctionnalités développées
5. Technologies utilisées
6. Solution applicative (concept, détails, difficultés…)
7. Infrastructure de l'application
8. Déploiement sur Rasberry PI
9. État actuel de l'application
10. Potentielles améliorations

1. <u>Introduction</u>

En 2015, après la bataille de Novi Grad, la **D.I.P**. (**Défense Informatique Planétaire**) s'est constituée pour lutter contre la menace numérique d'**Ultron**, capable de contaminer et détourner les réseaux mondiaux. Dans ce cadre, **nous — l'équipe Vision, une cellule spécialisée de la D.I.P. — avons été chargés de concevoir un outil de communication fiable, autonome et sécurisé**.
Notre mission : permettre à des communautés locales de continuer à échanger, s'organiser et partager des connaissances sans dépendre d'infrastructures compromises par Ultron.
De cette mission est née l'application **Vision**, une plateforme hébergée sur Raspberry Pi intégrant :

- un système d'authentification sécurisé,
- un forum / chat pour échanger en temps réel,
- un wiki pour documenter et capitaliser les savoirs.

L'objectif est de démontrer qu'**une solution robuste, simple à déployer et résiliente** peut soutenir la coordination et la survie d'équipes locales (et à terme ; nationale puis mondiale) dans un monde où les réseaux globaux ne sont plus sûrs d'utilisation.

2. Objectifs du projet

- Authentification sécurisée pour les utilisateurs
- Fournir un forum/chat pour discuter en temps réel.
- Mettre en place un wiki pour partager et organiser des connaissances.
- Héberger le tout sur un Raspberry Pi pour un déploiement simple et autonome.

3. Architecture du système

- **Serveur Raspberry Pi** : héberge l'application Flask et la base de données PostgreSQL.
- **Application Flask** : gère l'authentification, le forum et le wiki.
- **Base PostgreSQL** : stocke utilisateurs, messages et articles.
- **Frontend HTML/CSS** : interface utilisateur via templates Jinja2.

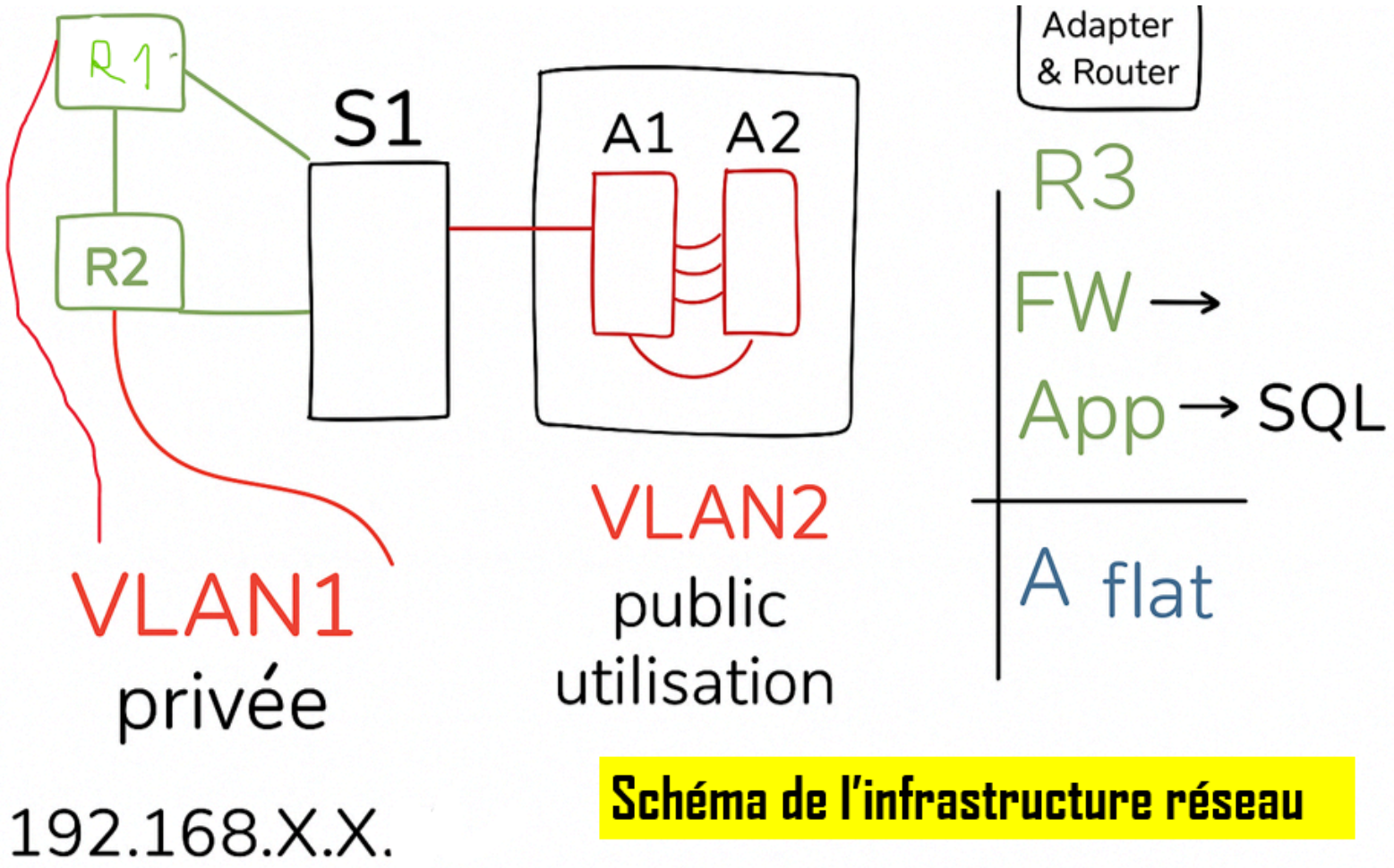


Schéma de l'infrastructure réseau

4. Fonctionnalités développées

1. **Authentification sécurisée**
    - Inscription et connexion avec mots de passe hashés (bcrypt).
    - Sessions utilisateur pour conserver la connexion.
2. **Forum / Chat**
    - Envoi et réception de messages.
    - Rafraîchissement automatique des conversations (AJAX).
    - Stockage des messages en base.
3. **Wiki**
    - Création et lecture d’articles.
    - Organisation de contenus utiles pour les utilisateurs.

5. Technologies utilisées

- **Backend** : Python (Flask, Jinja2…)

- **Base de données** : PostgreSQL.
  - **API** : PostgREST
  - **Adaptateur Python -> SQL**: Psycopg2

- **Sécurité** : Bcrypt pour le hachage des mots de passe.

- **Borne wifi** : Cisco & openWRT

- **Frontend** : HTML, CSS, Flask (Jinja2, Templates…)

- **Déploiement** : Raspberry Pi avec Linux (ssh)

6. Solution applicative

L’utilisation de Flask (Microframework Web) nous a permis de gérer beaucoup de choses sans avoir la nécessité de passer par des dépendances de manière excessive ou simplement d’avoir recours à plusieurs outils différents.

Pendant notre développement nous avons choisi a de multiples reprises de nous réorienter sur la manière de produire l’application :

Initialement nous étions parti sur l’idée (que nous avons gardés) d’une application qui tournerait sur nos raspberry PI et qui fonctionnerait de manière autonome en locale.

Cependant, c’est lors du choix des outils que nous avons évoqué plusieurs possibilités. Une première a été de faire une application sur Android Studio mais pour cause de non compatibilité avec nos autres outils (exemple : pfsense n’était pas compatible alors nous avons dû trouver d’autres alternatives, et il en as été de même pour Android Studio, le modèle et service de base de données que nous allions utiliser, et ainsi de suite jusqu'à notre configuration actuelle.)

Ainsi nous en sommes arrivés à notre configuration actuelle n’étant néanmoins pas encore optimisée et totalement aboutie. L’application en tant que tel est fonctionnelle mais n’est pas pleinement aboutie (gestion de la base de données, des adresses mac, front-end)

7. Infrastructure de l’application

/home/vision/1

```
├── app.py
├── requirements.txt
├── schema.sql
├── .env
├── templates/
│   ├── base.html
│   └── login.html
│   └── register.html
│   └── menu.html
│   └── forum.html
│   └── wiki.html
│   └── article.html
└── static/
    └── css/style.css
```

8. Déploiement sur Raspberry Pi

1. Copier les fichiers sur le Raspberry

- Prenez votre projet (le dossier `1/` avec `app.py`, `templates/`, `static/`, `requirements.txt`).
- Copiez-le **dans `/etc/1/`** sur le Raspberry Pi.
- Pour ça, vous pouvez utiliser :
  - Une **clé USB**
  - **scp** (si vous êtes en réseau : `scp -r 1/ pi@192.168.1.xxx:/etc/1/`)
  - Ou tout autre moyen de transfert de fichiers.

Résultat attendu :

```
/etc/1/app.py
/etc/1/templates/...
/etc/1/static/...
/etc/1/requirements.txt
```

2. Installer les dépendances

- Sur le Raspberry, placez-vous dans le dossi `cd /etc/1`
- Installez les dépendances listées dans `requirements.txt` : `pip install -r requirements.txt`

3. Lancer l'application pour tester

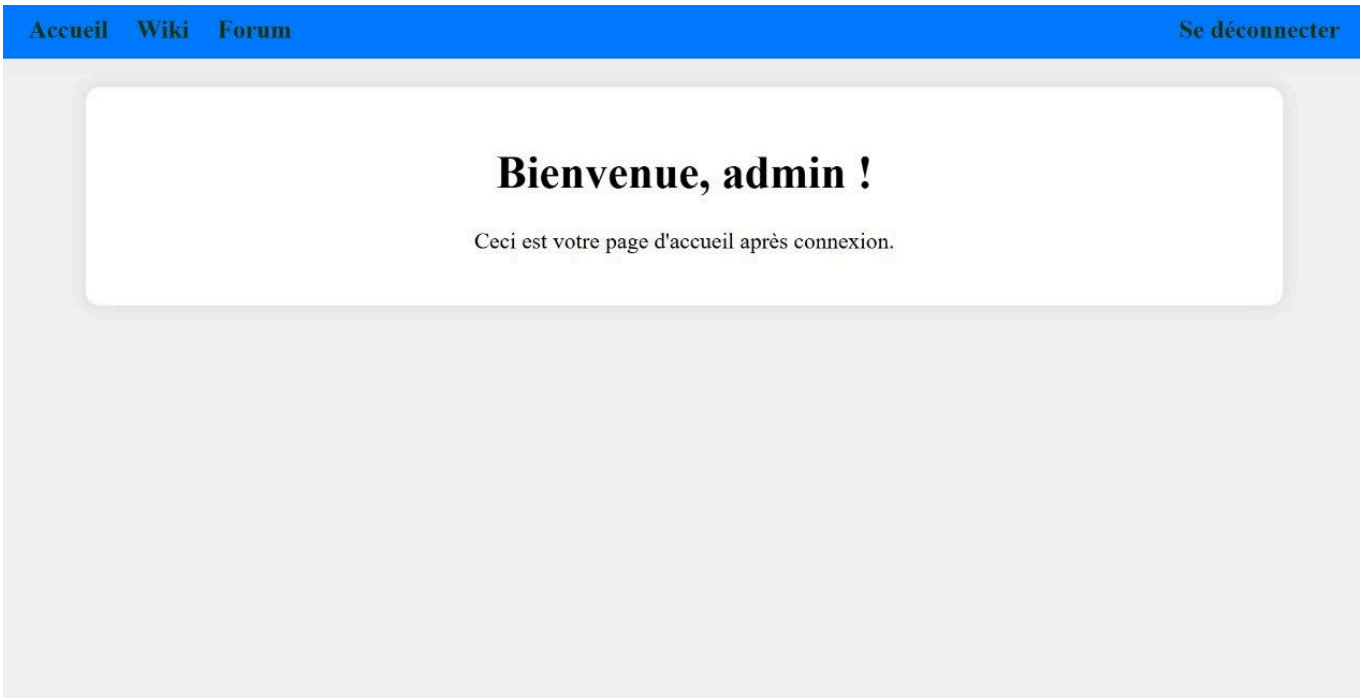
- Toujours dans `/etc/1/` : `python3 app.py`

  👉 À ce stade, l'app sera dispo depuis votre réseau local sur : `http://<ip_du_raspberry>:5000`

9. État actuel de l’application :

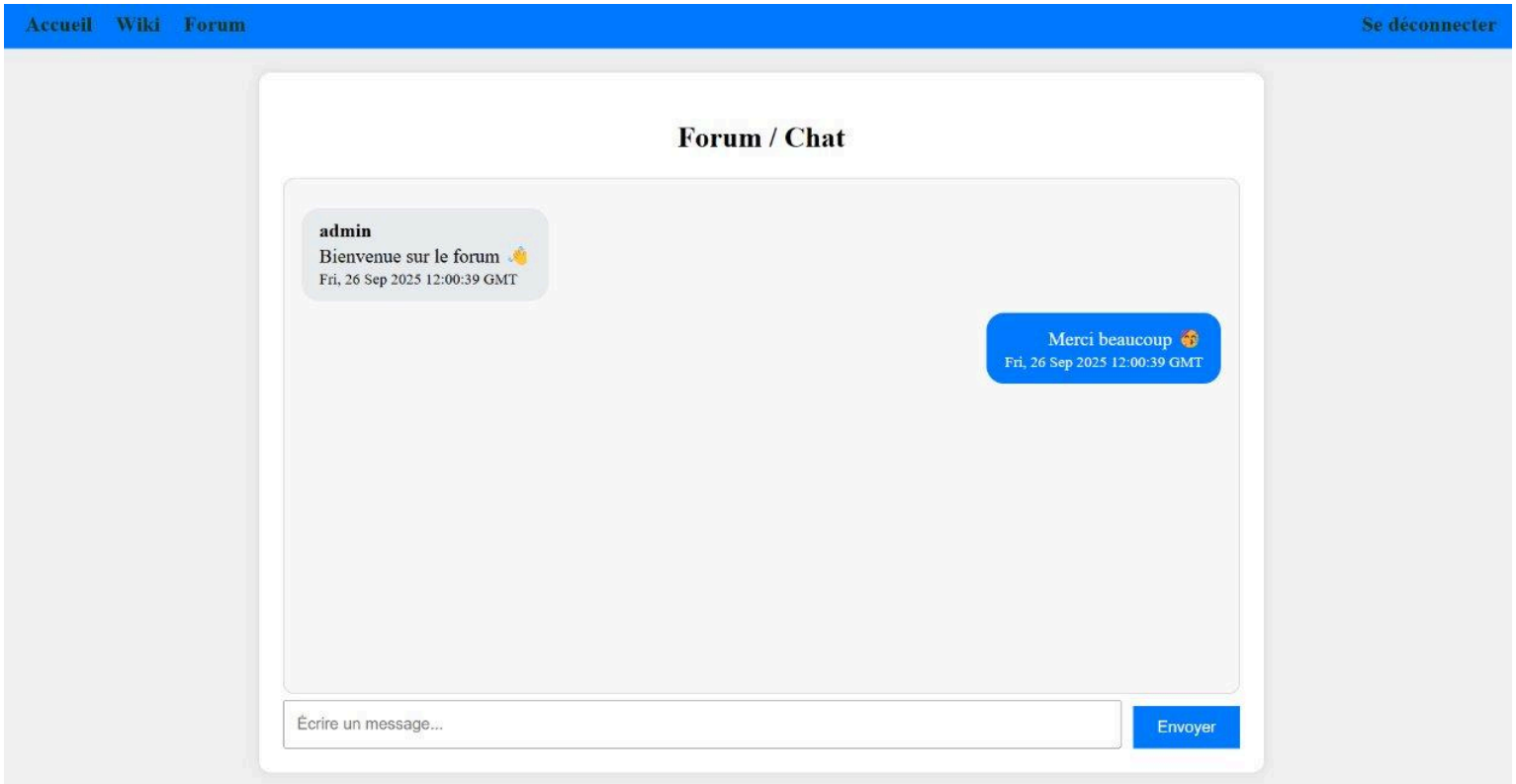
Page de connexion :

Page d’accueil :



Page du Wiki :

Page du Forum :



10. Potentielles améliorations :

- **Whitelist MAC :**
  - L'objectif aurait été de faire une gestion des autorisations au niveau des adresses mac dès lors qu'un appareil se connecte sur notre wifi établi afin de pouvoir potentiellement bloquer l'accès à des utilisateurs malveillants.

- **Map & Alertes :**
  - L'objectif de cette fonctionnalité aurait été d'implémenter une nouvelle page MAP avec l'utilisation de leaflet.js (open source) afin de pouvoir permettre aux utilisateurs de pouvoir placer des épingles à des endroits où la connexion est sécurisée, pouvoir mettre en évidence les points de résistances [...] tout ça gérer localement afin d'avoir un rempart supplémentaire face à Ultron.

- **Backups automatiques:**
  - L'objectif serait de pouvoir faire des backups de notre application à des intervalles réguliers afin de pouvoir pallier à des coupures réseaux et de ne pas être mis à mal si aucunes sauvegardes n'a été réalisées.

- **Modération & rôles :**
  - Il s'agissait d'implémenter des droits aux utilisateurs, par défaut un utilisateurs peut consulter et créer des articles sur le wiki et peut aussi échanger sur le forum de discussion instantanée ; et par extension il est logique de devoir implémenter des droits d'administrateur afin de pouvoir contrôler les utilisateurs ainsi que le contenu des articles et du forum.